SAT Encoding Approaches for Exact Sudoku Solving: A Comparison with ILP and CP Methods

Nguyen Duc Khiem[a]

[a] *Faculty of Information Technology, VNU University of Engineering and Technology, Ha Noi, Viet Nam*

ARTICLE INFO

Keywords:
Sudoku
SAT-based methods

ABSTRACT

Sudoku is a well-known puzzle played on an $n^2 \times n^2$ grid divided into $n \times n$ subgrids, where each row, column, and subgrid must contain every value exactly once. It can be written as a Boolean satisfiability (SAT) problem, and the main design choice is how to encode the *at-most-one* (AMO) constraints. In this paper, I compare five AMO encodings, namely pairwise, sequential, commander, product, and bitwise, implemented with the PySAT toolkit, on Sudoku grids from 4×4 up to 81×81. I then compare the best SAT configurations with Gurobi, CPLEX MIP, CPLEX CP, and OR-Tools CP-SAT on larger grids, up to 100×100. The results show that the pairwise encoding works well on small grids (4×4 and 9×9), but its number of clauses grows quickly with the grid size. From 16×16 to 81×81, the product encoding is the best choice. In the comparison with the other solvers, the SAT-based solver with the product encoding is still the fastest on most of the larger grids, but at 100×100 OR-Tools CP-SAT becomes faster, by about 30.31 ms on average. These results suggest that the choice of AMO encoding matters more as the grid grows, and that SAT remains competitive with ILP and CP solvers on Sudoku up to very large grids.

1. Introduction

Constraint Satisfaction Problems (CSPs) are a class of combinatorial search problems (a branch of optimization) in which variables are assigned values from valid domains to satisfy a set of given constraints. In practice, problems involving constraints and conflict avoidance are often modeled using this framework and solved with specialized algorithms; examples include scheduling football matches so that no team plays twice in a single week or goes a full week without a match, and arranging work schedules so that no individual is assigned to two tasks simultaneously.

Sudoku is a classic example of this type of problem. A Sudoku puzzle consists of an $n^2 \times n^2$ square grid, partially filled with numbers from 1 to n^2; players must complete the remaining cells such that no number repeats within the same row or column, while the numbers in each $n\times n$ subgrid must also satisfy the same condition. Various approaches exist for tackling this problem, including CP-based [1], ILP-based [2], and SAT-based [3, 4].

However, most existing studies on SAT-based Sudoku solving apply a single, fixed *at-most-one* (AMO) encoding scheme—typically the pairwise encoding—without systematically evaluating how alternative cardinality-constraint encodings, such as the sequential [5], commander [6], product [7], or bitwise [8], affect solving performance on the same benchmark [4]. Moreover, while SAT solvers have been shown to be competitive exact methods for combinatorial puzzles [3], few studies directly compare SAT-based Sudoku solvers against modern commercial integer-programming and constraint-programming solvers, such as Gurobi [9], CPLEX MIP [10], CPLEX CP [10], and CP-SAT [11], under a unified experimental setting. As a result, it remains unclear which AMO/ALO encoding is most effective for Sudoku, and how SAT-based solving compares to these alternative exact paradigms in practice.

In this paper, I resolve this gap by (i) implementing five AMO encoding schemes—pairwise, sequential , commander, product, and bitwise—for the cell, row, column, and block constraints of Sudoku using PySAT; (ii) benchmarking the resulting SAT-based solver against Gurobi, CPLEX MIP, CPLEX CP, and CP-SAT under identical conditions; and (iii) evaluating scalability across multiple grid sizes. Section 2 presents the problem formulation and related work; Section 3 describes the proposed encoding; Section 4 reports results; and Section 5 concludes the paper.

nguyenkhiem1812.206@gmail.com (N.D. Khiem)
ORCID(s): 0009-0005-6212-4396 (N.D. Khiem)

2. Background

2.1. Problem Definition and ILP Formulation

A Sudoku puzzle is defined on an $n^2 \times n^2$ square grid, partitioned into $n \times n$ subgrids of size $n \times n$ each. I define the binary decision variable:

$$a_{i,j,v} \in \{0,1\} \text{ with } i,j,v \in \{1,\dots,n\}, \text{ where } a_{i,j,v} = 1 \iff \text{cell } (i,j) \text{ is assigned value } v.$$

Feasibility problem. Since a well-posed Sudoku puzzle has a unique solution, the ILP formulation reduces to a feasibility problem: we are only interested in finding an assignment of $a_{r,c,v}$ that satisfies all constraints, rather than optimizing any quantity [12]. Formally, I set the objective function to the zero function,

$$f(a_{1,1,1}, a_{1,1,2}, ..., a_{n,n,n^2}) = \min\ 0,$$

so that any feasible solution to the constraints below is considered optimal.

Constraints.

$$\sum_{v=1}^{n} a_{i,j,v} = 1, \qquad \forall i,j \in \{1,\dots,n\} \tag{1}$$

$$\sum_{c=1}^{n} a_{i,j,v} = 1, \qquad \forall i,v \in \{1,\dots,n\} \tag{2}$$

$$\sum_{r=1}^{n} a_{i,j,v} = 1, \qquad \forall j,v \in \{1,\dots,n\} \tag{3}$$

$$\sum_{(i,j)\in S_k} a_{r,c,v} = 1, \qquad \forall k,v \in \{1,\dots,n\} \tag{4}$$

$$a_{i,j,v} = 1, \qquad \forall (i,j,v) \in G \tag{5}$$

where S_k denotes the set of cells belonging to subgrid k, and G is the set of given (numbered) cell-value assignments in the input puzzle.

2.2. Related Work

Integer Linear Programming (ILP). Sudoku can also be formulated as a binary integer linear program and solved with general-purpose MIP solvers. Bartlett et al. [2] model each cell-value pair as a binary variable and express the row, column, and subgrid rules as linear equality constraints, treating the puzzle as a feasibility problem that needs no objective function. This is the same formulation we use in Section 2.1. The model is easy to state and can be passed directly to commercial solvers such as Gurobi and CPLEX, but it is not specific to Sudoku and does not exploit the logical structure of the constraints.

Constraint Programming (CP). Another approach involves directly modeling the Sudoku problem as a constraint satisfaction problem and solving it using general-purpose CP solvers. Musliu and Winter [1] proposed a hybrid method combining constraint propagation techniques with local search to enhance solving efficiency for difficult instances. Although these CP-based methods take advantage of powerful existing propagation techniques (e.g., the AllDifferent constraint [13]), their performance depends on the search heuristics of specific solvers; consequently, direct comparisons between different implementations are not always feasible. In this paper, we use two widely available CP solvers: CPLEX CP [10] and Google OR-Tools CP-SAT [11], the latter combining constraint propagation with SAT-style clause learning.

SAT-based methods. Boolean satisfiability offers a natural alternative for encoding Sudoku's row, column, block, and cell constraints as CNF formulas. Weber [3] and Lynce and Ouaknine [4] were among the first to formulate Sudoku as a SAT instance, reporting that SAT solvers can outperform constraint-programming approaches on large or hard puzzles. However, these early SAT-based solvers rely on a single, fixed *at-most-one* (AMO) encoding scheme (typically pairwise), without exploring alternative cardinality-constraint encodings developed in the broader SAT literature, such as the sequential [5], commander [6], product [7], and bitwise [8] encodings—each offering different trade-offs in the number of auxiliary variables and clauses.

Overall, prior work on Sudoku solving has largely treated ILP-based, CP-based, and SAT-based approaches in isolation, and within the SAT-based line of work, existing studies have not systematically compared multiple AMO/ALO encoding schemes, nor benchmarked them against modern commercial ILP and CP solvers such as Gurobi, CPLEX, and CP-SAT under a unified experimental setting. This paper aims to fill this gap.

3. SAT-Based Encoding for Sudoku

3.1. Encoding Overview

I encode the ILP formulation introduced in Section 2.1 as a Boolean satisfiability (SAT) instance. For each cell (i, j) and value v, I introduce a Boolean variable $a_{i,j,v} \in \{\text{TRUE}, \text{FALSE}\}$, directly corresponding to the binary decision variable $a_{i,j,v} \in \{0, 1\}$ defined earlier, where $a_{i,j,v} = \text{TRUE}$ if and only if cell (i, j) is assigned value v. Each linear equality constraint of the form $\sum a_{i,j,v} = 1$ in the ILP formulation—namely the cell (Eq. 1), row (Eq. 2), column (Eq. 3), and subgrid (Eq. 4) constraints—translates into an *exactly-one* constraint over the corresponding group of Boolean variables, which I decompose into two CNF-encodable components: an *at-least-one* (ALO) clause, ensuring that at least one variable in the group is TRUE, and an *at-most-one* (AMO) constraint, ensuring that no two variables in the group are simultaneously TRUE. The given-cell constraints (Eq. 5) are encoded directly as unit clauses. Since Sudoku is a decision problem rather than an optimization problem with a bound to tighten (unlike, e.g., bandwidth minimization problem [14]), a single SAT query encoding all constraints suffices, and no incremental solving procedure is required.

The central technical focus of this work lies not in the solving procedure itself, but in the choice of encoding for the *at-most-one* constraints that arise in the cell, row, column, and subgrid constraint groups. While the *at-least-one* component admits a single natural CNF encoding (a disjunctive clause over the group), the *at-most-one* component can be encoded in several inequivalent ways, differing in the number of auxiliary variables and clauses they introduce. Section 3.2 presents and compares five such AMO encoding schemes—pairwise, sequential , commander, product, and bitwise—applied uniformly across all four constraint groups, in order to assess their effect on solving performance under an identical constraint structure.

3.2. At-Most-One (AMO) Encoding Variants

To keep the notation general and applicable uniformly to all four constraint groups (cell, row, column, grid/subgrid), I describe each AMO encoding scheme over a generic set of n Boolean variables $x_1, \ldots, x_n$; in our application, this set corresponds to the variables $a_{i,j,v}$ ranging over the appropriate index (e.g., $x_v = a_{i,j,v}$ for $v = 1, \ldots, n$ when encoding the cell constraint at position (i, j)).

Pairwise encoding. The pairwise encoding is the most direct formulation of the *at-most-one* constraint: it states that for every pair of variables in the group, at least one of the two must be FALSE, thereby forbidding both from being TRUE simultaneously [8].

- *Definition.* Given variables $x_1, \ldots, x_n$, the *at-most-one* constraint is satisfied if and only if

$$\bigwedge_{1 \le i < j \le n} (\neg x_i \vee \neg x_j)$$

- *Auxiliary variables.* None. The pairwise encoding introduces no additional Boolean variables beyond $x_1, \ldots, x_n$.
- *CNF clauses.* Each pair (i, j) with $i < j$ contributes exactly one binary clause $(\neg x_i \vee \neg x_j)$. The full encoding therefore consists of $\binom{n}{2} = \frac{n(n-1)}{2}$ clauses, each of length 2.

Bitwise encoding. The bitwise encoding takes a fundamentally different approach from the pairwise encoding: instead of directly forbidding every pair of variables from being TRUE simultaneously, it introduces auxiliary variables that encode, in binary, the index of the (*at most one*) variable allowed to be TRUE [8].

- *Definition.* Given variables $x_1, \ldots, x_n$, let $m = \lceil \log_2 n \rceil$. I introduce auxiliary Boolean variables $y_1, \ldots, y_m$, which jointly encode, as a binary number, the index of the variable x_k that is TRUE (if any). For each variable x_k ($k = 1, \ldots, n$) and each bit position $b = 1, \ldots, m$, let $\text{bit}(k, b)$ denote the b-th bit of the binary representation of k. The *at-most-one* constraint is enforced by requiring that whenever x_k is TRUE, the auxiliary variables $y_1, \ldots, y_m$ must match the binary encoding of $k - 1$:

$$\bigwedge_{k=1}^{n}\bigwedge_{b=1}^{m}\left(\neg x_k \vee c_{k,b}\right), \quad \text{where } c_{k,b} = \begin{cases} y_b & \text{if bit}(k-1,b)=1 \\ \neg y_b & \text{if bit}(k-1,b)=0. \end{cases}$$

If two variables x_i and x_j $(i \neq j)$ were both TRUE, the clauses above would force $y_1, \ldots, y_m$ to match two distinct binary encodings simultaneously, which is unsatisfiable; hence at most one x_k can be TRUE.

- *Auxiliary variables.* $m = \lceil \log_2 n \rceil$ Boolean variables $y_1, \ldots, y_m$.
- *CNF clauses.* Each variable x_k contributes exactly m binary clauses (one per bit position), so the encoding consists of $n \cdot m = n\lceil \log_2 n \rceil$ clauses in total, each of length 2.

Commander encoding. The commander encoding partitions the variables into small groups, introduces one auxiliary *commander* variable per group to represent whether that group contains the TRUE variable, and then recursively enforces the *at-most-one* constraint over the (much smaller) set of commander variables [6].

- *Definition.* Given variables $x_1, \ldots, x_n$, partition them into $g = \left\lceil \frac{n}{c} \right\rceil$ groups $G_1, \ldots, G_g$, each of constant size c (commonly $c = 3$) [6]. For each group $G_i = \{x_{i,1}, \ldots, x_{i,c}\}$, introduce a commander variable d_i, and impose:

$$\underbrace{\text{AMO}(G_i)}_{\substack{\textit{at-most-one} \text{ within } G_i \\ \text{(encoding scheme optional}*\text{)}}} \wedge \underbrace{(\neg d_i \vee x_{i,1} \vee \cdots \vee x_{i,c})}_{d_i \text{ is TRUE} \Rightarrow \exists j : x_{i,j} \text{ is TRUE}} \wedge \underbrace{\bigwedge_{j=1}^{c}(d_i \vee \neg x_{i,j})}_{x_{i,j} \text{ is TRUE} \Rightarrow d_i \text{ is TRUE}}$$

for each group $i = 1, \ldots, g$, and then recursively apply the same construction to the commander variables $d_1, \ldots, d_g$ in place of $x_1, \ldots, x_n$, until at most one commander variable remains at the current level, at which point the *at-most-one* constraint is trivially satisfied and no further clauses are added. Together, these clauses ensure that at most one commander d_i can be TRUE, that a group's commander is TRUE if and only if one of its variables is TRUE, and that at most one variable within a TRUE-commander group can be TRUE—hence at most one x_k overall can be TRUE.

*The *at-most-one* constraint within each group, AMO(G_i), may itself be encoded using any of the AMO encoding discussed in this section (pairwise [8], sequential [5], product [7], bitwise [8]); the choice is implementation-dependent and does not affect the correctness of the recursive construction, only its constant factors. In our implementation, I apply the pairwise encoding for AMO(G_i), since each group has constant size $c = 3$, making the $\binom{c}{2} = 3$ pairwise clauses per group negligible in cost while keeping the construction simple to implement and free of additional auxiliary variables [6].

- *Auxiliary variables.* One commander variable d_i per group at each recursion level, plus any auxiliary variables required by the chosen encoding of AMO(G_i) within each group. At the first level, the number of commander variables is $g = \left\lceil \frac{n}{c} \right\rceil$; since each subsequent level reduces the variable count by the constant factor c, the total number of commander variables across all levels sums geometrically to $\mathcal{O}\left(\frac{n}{c}\right) = \mathcal{O}(g)$. In practice, AMO$(G_i)$ is commonly instantiated with the pairwise encoding, since each group has small constant size c, which keeps the construction simple and requires no additional auxiliary variables per group [6]; under this common choice, the total number of auxiliary variables is dominated by the commander variables themselves, i.e., $\mathcal{O}\left(\frac{n}{c}\right)$.
- *CNF clauses.* The total number of clauses depends on both the number of groups g and the encoding chosen for AMO(G_i) within each group: each group contributes the clauses of its chosen per-group AMO encoding, plus one clause of length $c+1$ and c implication clauses linking d_i to its group. Under the common choice of pairwise encoding for each group (constant size c) [6], each group contributes a constant number of clauses, so a single recursion level over n variables contributes $\mathcal{O}(n)$ clauses; summed geometrically over the $\mathcal{O}(\log n)$ recursion levels, the total remains $\mathcal{O}(n)$.

Sequential encoding. The sequential encoding introduces a chain of auxiliary *register* variables that track, one variable at a time, whether at least one of the preceding variables has already been set to TRUE; once the chain registers a TRUE, no subsequent variable is permitted to be TRUE [5].

- *Definition.* Given variables $x_1, \dots, x_n$, introduce auxiliary Boolean variables $c_1, \dots, c_{n-1}$, where c_i is intended to represent whether at least one of $x_1, \dots, x_i$ is TRUE. The construction is governed by three rules:

  Rule 1. If x_i is TRUE, then c_i must be TRUE, for $i = 1, \dots, n-1$:

  $$x_i \Rightarrow c_i \quad \Longleftrightarrow \quad (\neg x_i \vee c_i), \qquad i = 1, \dots, n-1.$$

  Rule 2. If c_{i-1} is TRUE, then c_i must also be TRUE, for $i = 2, \dots, n-1$:

  $$c_{i-1} \Rightarrow c_i \quad \Longleftrightarrow \quad (\neg c_{i-1} \vee c_i), \qquad i = 2, \dots, n-1.$$

  Rule 3. If c_{i-1} is TRUE, then x_i must be FALSE, for $i = 2, \dots, n$:

  $$c_{i-1} \Rightarrow \neg x_i \quad \Longleftrightarrow \quad (\neg x_i \vee \neg c_{i-1}), \qquad i = 2, \dots, n.$$

  The *at-most-one* constraint is then enforced by the conjunction of all three rules:

  $$\bigwedge_{i=1}^{n-1} (\neg x_i \vee c_i) \;\wedge\; \bigwedge_{i=2}^{n-1} (\neg c_{i-1} \vee c_i) \;\wedge\; \bigwedge_{i=2}^{n} (\neg x_i \vee \neg c_{i-1}).$$

  Intuitively, once some x_i is TRUE, Rule 1 sets c_i to TRUE, Rule 2 propagates this TRUE register forward through $c_{i+1}, \dots, c_{n-1}$, and Rule 3 forbids every subsequent $x_{i+1}, \dots, x_n$ from being TRUE—hence at most one x_k overall can be TRUE.

- *Auxiliary variables.* $n-1$ Boolean register variables $c_1, \dots, c_{n-1}$, i.e., $\mathcal{O}(n)$ auxiliary variables.

- *CNF clauses.* The encoding consists of 1 clause of the first type, $n-2$ clauses of the second type, $n-2$ clauses of the third type, and $n-1$ clauses of the fourth type, for a total of $1 + (n-2) + (n-2) + (n-1) = 3n-4$ clauses, i.e., $\mathcal{O}(n)$ clauses, each of length at most 2.

Product encoding. The product encoding arranges the n variables into a two-dimensional grid indexed by a row and a column, introduces one auxiliary variable per row and one per column to indicate whether the TRUE variable (if any) lies in that row or column, and enforces the *at-most-one* constraint by requiring at most one row variable and at most one column variable to be TRUE [7].

- *Definition.* Given variables $x_1, \dots, x_n$, choose p, q such that $p \cdot q \geq n$, commonly $p = q = \lceil \sqrt{n} \rceil$ [7], and re-index each variable as $x_{i,j}$ for $i = 1, \dots, p$ and $j = 1, \dots, q$ (with any surplus positions, if $p \cdot q > n$, left unused). Introduce auxiliary Boolean variables $r_1, \dots, r_p$ (row selectors) and $c_1, \dots, c_q$ (column selectors). The *at-most-one* constraint is enforced by the conjunction:

  $$\underbrace{\bigwedge_{i,j} \left[(\neg x_{i,j} \vee r_i) \wedge (\neg x_{i,j} \vee c_j) \right]}_{\text{if } x_{i,j} \text{ is TRUE} \Rightarrow r_i \text{ and } c_j \text{ both TRUE}} \;\wedge\; \mathrm{AMO}(r_1, \dots, r_p) \;\wedge\; \mathrm{AMO}(c_1, \dots, c_q),$$

  where $\mathrm{AMO}(r_1, \dots, r_p)$ and $\mathrm{AMO}(c_1, \dots, c_q)$ denote the *at-most-one* constraint over the row and column selectors, respectively, each encoded using any of the AMO encoding discussed in this section (the choice is implementation-dependent, similarly to the choice of $\mathrm{AMO}(G_i)$ within each group of the commander encoding in Section 3.2; a natural choice is the pairwise encoding, since $p = q = \mathcal{O}(\sqrt{n})$ is small [7]). Intuitively, if two variables x_{i_1,j_1} and x_{i_2,j_2} were both TRUE, then either $i_1 = i_2$ (same row, forcing r_{i_1} and r_{i_2} both TRUE, violating $\mathrm{AMO}(r_1, \dots, r_p)$) or $j_1 = j_2$ (same column, violating $\mathrm{AMO}(c_1, \dots, c_q)$) or both differ (in which case r_{i_1}, r_{i_2} both TRUE, again violating $\mathrm{AMO}(r_1, \dots, r_p)$)—hence at most one $x_{i,j}$ overall can be TRUE.

- *Auxiliary variables.* p row selectors and q column selectors, i.e., $p + q$ auxiliary variables. With $p = q = \lceil \sqrt{n} \rceil$, this is $2\lceil \sqrt{n} \rceil$ auxiliary variables, i.e., $\mathcal{O}(\sqrt{n})$.
- *CNF clauses.* The implication group $(x_{i,j} \Rightarrow r_i \wedge c_j)$ contributes $2n$ clauses (two per variable $x_{i,j}$); the row-pair prohibitions contribute $\binom{p}{2}$ clauses; the column-pair prohibitions contribute $\binom{q}{2}$ clauses. With $p = q = \lceil \sqrt{n} \rceil$, the total is

$$2n + \binom{p}{2} + \binom{q}{2} \approx 2n + \frac{p^2}{2} + \frac{q^2}{2} \approx 2n + \frac{n}{2} + \frac{n}{2} = 3n$$

clauses, i.e., $\mathcal{O}(n)$.

Table 1. Comparison of the number of auxiliary variables and clauses by the previously mentioned encoding methods.

Encoding	Auxiliary variables	Clauses
Pairwise	0	$\mathcal{O}(n^2)$
Bitwise	$\mathcal{O}(\log(n))$	$\mathcal{O}(n \log(n))$
Commander	$\mathcal{O}\left(\frac{n}{c}\right)$	$\mathcal{O}(n)$
Sequential	$\mathcal{O}(n)$	$\mathcal{O}(n)$
Product	$\mathcal{O}(\sqrt{n})$	$\mathcal{O}(n)$

Table 2. Comparison of the number of auxiliary variables and clauses for each encoding method, applied to a Sudoku grid of size $n^2 \times n^2$, each constraint group - row, column, subgrid has n^2 Boolean variables.

Encoding	Auxiliary variables	Clauses
Pairwise	0	$\mathcal{O}(n^4)$
Bitwise	$\mathcal{O}(\log(n))$	$\mathcal{O}(n^2 \log(n))$
Commander	$\mathcal{O}\left(\frac{n^2}{c}\right)$	$\mathcal{O}(n^2)$
Sequential	$\mathcal{O}(n^2)$	$\mathcal{O}(n^2)$
Product	$\mathcal{O}(n)$	$\mathcal{O}(n^2)$

3.3. At-Least-One (ALO) Encoding

Unlike the *at-most-one* constraint, which admits several inequivalent encoding schemes as discussed in Section 3.2, the *at-least-one* constraint over a group of variables $x_1, \ldots, x_n$ admits a single natural CNF encoding: a disjunctive clause over all variables in the group,

$$x_1 \vee x_2 \vee \cdots \vee x_n,$$

which is satisfied if and only if at least one x_k is TRUE. This encoding requires no auxiliary variables and contributes exactly one clause of length n per constraint group, regardless of which AMO encoding scheme is subsequently combined with it.

For each of the four constraint groups (cell, row, column, and subgrid), the corresponding *exactly-one* constraint is obtained by conjoining this single ALO clause with the AMO encoding of the same group, using whichever of the five schemes described in Section 3.2 (pairwise, sequential, commander, product, or bitwise) is selected for that group:

$$\underbrace{(x_1 \vee \cdots \vee x_n)}_{\textit{at-least-one}} \wedge \underbrace{\mathrm{AMO}(x_1, \ldots, x_n)}_{\textit{at-most-one}\text{ - any scheme}}.$$

Since the ALO clause is independent of the AMO scheme chosen, it is generated identically regardless of which of the five AMO encodings is applied, allowing a fair, like-for-like comparison across encoding schemes in the experiments of Section 4: only the AMO component varies, while the ALO component remains fixed throughout.

3.4. Solving Scheme

Algorithm 1 summarizes our implementation, following the indexing convention used in the codebase: for an $n^2 \times n^2$ grid (letting $\texttt{size} = n^2$), each variable $a_{i,j,v}$ is mapped to a unique integer literal identifier via

$$\texttt{get_id(i, j, v)} = \texttt{i} \cdot \texttt{size}^2 + \texttt{j} \cdot \texttt{size} + \texttt{v}, \qquad \texttt{i, j} \in \{0, \ldots, \texttt{size} - 1\},\ \texttt{v} \in \{1, \ldots, \texttt{size}\},$$

and each of the four constraint groups is encoded via a shared `add_exactly_one` routine, which first appends the ALO clause and then invokes the chosen AMO scheme's encode procedure to append the corresponding AMO clauses, allocating any required auxiliary variable identifiers starting from $\texttt{current_id} = \texttt{size}^3$ (i.e., past the range used by the original variables). For the subgrid constraint, each of the n^2 subgrids is indexed by a pair $(\texttt{sr}, \texttt{sc})$, and the cell at local offset $(\texttt{or}, \texttt{oc})$ within that subgrid corresponds to the global grid position $(\texttt{sr}\cdot\texttt{n}+\texttt{or},\ \texttt{sc}\cdot\texttt{n}+\texttt{oc})$. The SAT solver used to solve the resulting formula is treated as an explicit parameter of the procedure, alongside the AMO scheme, allowing both to be varied independently in the experiments of Section 4.

Algorithm 1 SAT-based Sudoku solving scheme implementation.

Require: Puzzle grid `B` of size: $\texttt{size} \times \texttt{size}$ (0 denotes a blank cell); order `n` ($\texttt{size} = \texttt{n}^2$); AMO encoding scheme `E`; SAT solver name `name_solver`

Ensure: Result (SAT or UNSAT), number of clauses, number of auxiliary variables, and solving time

$\texttt{R} \leftarrow \emptyset$; $\texttt{current_id} \leftarrow \texttt{size}^3$ ▷ Largest identifier used by the main variables; auxiliary variables get larger identifiers

function ADD_EXACTLY_ONE(L) ▷ L: list of literal ids for one constraint group

  $\texttt{R} \leftarrow \texttt{R} \cup \{\texttt{L}\}$ ▷ ALO clause

  $\texttt{C}, \texttt{current_id} \leftarrow \texttt{E.encode(L, current_id)}$ ▷ AMO clauses, per Section 3.2

  $\texttt{R} \leftarrow \texttt{R} \cup \texttt{C}$

end function

for $\texttt{i, j} \in \{0, \ldots, \texttt{size} - 1\}$ **do** ▷ Cell constraint

  ADD_EXACTLY_ONE($[\texttt{get_id(i, j, v)} : \texttt{v} = 1, \ldots, \texttt{size}]$)

end for

for $\texttt{j} \in \{0, \ldots, \texttt{size} - 1\}, \texttt{v} \in \{1, \ldots, \texttt{size}\}$ **do** ▷ Column constraint (fixed column `j`, varying row `i`)

  ADD_EXACTLY_ONE($[\texttt{get_id(i, j, v)} : \texttt{i} = 0, \ldots, \texttt{size} - 1]$)

end for

for $\texttt{i} \in \{0, \ldots, \texttt{size} - 1\}, \texttt{v} \in \{1, \ldots, \texttt{size}\}$ **do** ▷ Row constraint (fixed row `i`, varying column `j`)

  ADD_EXACTLY_ONE($[\texttt{get_id(i, j, v)} : \texttt{j} = 0, \ldots, \texttt{size} - 1]$)

end for

for $\texttt{sr, sc} \in \{0, \ldots, \texttt{n} - 1\}, \texttt{v} \in \{1, \ldots, \texttt{size}\}$ **do** ▷ Subgrid constraint

  ADD_EXACTLY_ONE($[\texttt{get_id(sr} \cdot \texttt{n+dr, sc} \cdot \texttt{n+dc, v)} : \texttt{dr, dc} = 0, \ldots, \texttt{n} - 1]$)

end for

for `i, j` such that $\texttt{B[i][j]} \neq 0$ **do** ▷ Given cells

  $\texttt{R} \leftarrow \texttt{R} \cup \{[\texttt{get_id(i, j, B[i][j])}]\}$ ▷ Unit clause

end for

$\texttt{S} \leftarrow$ new SAT solver named `name_solver`; add all clauses of `R` to `S`

$\texttt{num_clauses} \leftarrow |\texttt{R}|$; $\texttt{num_aux_vars} \leftarrow \texttt{current_id} - \texttt{size}^3$

$\texttt{t}_0 \leftarrow$ current time

$\texttt{sat} \leftarrow \texttt{S.solve()}$

$\texttt{solving_time} \leftarrow$ current time $- \texttt{t}_0$ ▷ Only the solving call is timed

return (SAT **if** `sat` **else** UNSAT), `num_clauses`, `num_aux_vars`, `solving_time` ▷ The grid can be decoded from the model, but this step is not executed in the experiments

I implemented this procedure using the PySAT toolkit, invoked via PySAT's Solver interface with $\texttt{name_solver} \in \{\texttt{Cadical195}, \texttt{Glucose4}, \texttt{MapleCM}, \texttt{Minisat22}, \texttt{Mergesat3}\}$. Since a well-posed Sudoku puzzle admits exactly one solution, a single SAT query per puzzle-encoding-solver combination suffices, and an unsatisfiable outcome is treated as a sanity check during development, as it indicates an error in the encoding rather than a property of the puzzle itself.

4. Computational Experiments

4.1. Experimental Setup

Software. I implement the SAT-based encodings in Python using the PySAT toolkit (version 1.9.dev5) [15]. In an initial comparison, I run five SAT solvers from PySAT, namely `Cadical195`, `Glucose4`, `MapleCM`, `Minisat22`, and `Mergesat3` (Figure 3). `Cadical195` is the fastest of the five, and `Minisat22` is the next fastest, with the smallest variation between instances. I therefore use these two solvers for the detailed ablation study in Section 4.2, and as the SAT solvers in the comparison with ILP and CP methods in Section 4.3, where the SAT side uses the product encoding, which performed best on the larger grids. For the comparison, I also solve the ILP formulation of Section 2.1 using Gurobi [9] and CPLEX MIP [10], and the CP formulation using CPLEX CP [10] and Google OR-Tools CP-SAT [11].

Dataset. I evaluate my approach on a set of Sudoku puzzles with many grid sizes, and I divide them into three groups for different test purposes. Specifically, Group S (4×4 to 25×25, 50 instances per size) is created using the `py-sudoku` library, while I collect Group M and Group L from the public dataset provided by [16]. I use Group S to compare the five AMO encodings together and to find the best encoding and solver combination when the grid size increases (Section 4.2). After that, I test this best combination against the ILP and CP solvers on Group M (36×36 to 64×64, 10 instances per size), where ILP and CP solvers can still solve the puzzles, but the speed of SAT-based methods becomes very important (Section 4.3). Finally, I use Group L to see how well the solvers and the encoding schemes perform on very large grids (Section 4.4). Table 3 summarizes the composition of each group.

Table 3. Composition of the Sudoku benchmark, partitioned by group.

Group	Grid sizes	Instances
S	4×4	50
	9×9	50
	16×16	50
	25×25	50
M	36×36	10
	49×49	10
	64×64	10
L	81×81	10
	100×100	10

Evaluation. Since a good Sudoku puzzle is generated to be feasible (SAT) and has a unique solution, no optimization solutions. I therefore evaluate each methods by solving time only. Each reported time is the average and median of the per-instance solving times in the raw CSV files, grouped by grid size, encoding, and solver.

4.2. Ablation Study of AMO Encodings

Table 4 and Table 5 report, for each AMO encoding and grid size, the average number of auxiliary variables and clauses generated, together with the average and median solving time, on Group S of the benchmark, using `Cadical195` and `Minisat22` respectively. As in Section 3.2, the number of auxiliary variables and clauses depends only on the encoding and grid size, not the solver, and matches exactly the theoretical formulas derived earlier.

Table 4. Average number of auxiliary variables, clauses, and solving time using `Cadical195` for each AMO encoding.

Grid size	Encoding	Avg. aux. vars	Avg. clauses	Avg. time (ms)	Median time (ms)
4×4	Pairwise	0	451.30	0.01	0.01
	Bitwise	128	579.30	0.01	0.01
	Commander	192	899.30	0.01	0.01
	Sequential	192	579.30	0.01	0.01
	Product	256	707.30	0.01	0.01
9×9	Pairwise	0	12007.40	0.08	0.07
	Bitwise	1296	12007.40	0.13	0.10
	Commander	1296	9415.40	0.18	0.15
	Sequential	2592	7795.40	0.17	0.14
	Product	1944	8119.40	0.14	0.12
16×16	Pairwise	0	124045.00	0.18	0.11
	Bitwise	4096	66701.00	0.16	0.13
	Commander	9216	57485.00	0.18	0.14
	Sequential	15360	46221.00	0.32	0.26
	Product	8192	46221.00	0.23	0.14
25×25	Pairwise	0	752909.00	4.69	6.73
	Bitwise	12500	315409.00	2.04	3.03
	Commander	32500	217909.00	1.85	2.52
	Sequential	60000	180409.00	1.91	2.39
	Product	25000	177909.00	1.44	1.83

Table 5. Average number of auxiliary variables, clauses, and solving time using `Minisat22` for each AMO encoding.

Grid size	Encoding	Avg. aux. vars	Avg. clauses	Avg. time (ms)	Median time (ms)
4×4	Pairwise	0	451.30	0.04	0.03
	Bitwise	128	579.30	0.04	0.04
	Commander	192	899.30	0.06	0.06
	Sequential	192	579.30	0.05	0.05
	Product	256	707.30	0.05	0.05
9×9	Pairwise	0	12007.40	0.25	0.26
	Bitwise	1296	12007.40	0.31	0.32
	Commander	1296	9415.40	0.40	0.41
	Sequential	2592	7795.40	0.38	0.38
	Product	1944	8119.40	0.29	0.30
16×16	Pairwise	0	124045.00	1.66	1.33
	Bitwise	4096	66701.00	1.08	0.97
	Commander	9216	57485.00	1.52	1.11
	Sequential	15360	46221.00	1.11	1.05
	Product	8192	46221.00	1.03	0.90
25×25	Pairwise	0	752909.00	10.36	9.77
	Bitwise	12500	315409.00	5.13	4.43
	Commander	32500	217909.00	5.23	4.49
	Sequential	60000	180409.00	4.60	4.03
	Product	25000	177909.00	3.70	3.36

Figure 1. Cumulative solving time by AMO encoding of `Cadical195`, across: (a) 4×4, (b) 9×9, (c) 16×16, (d) 25×25.

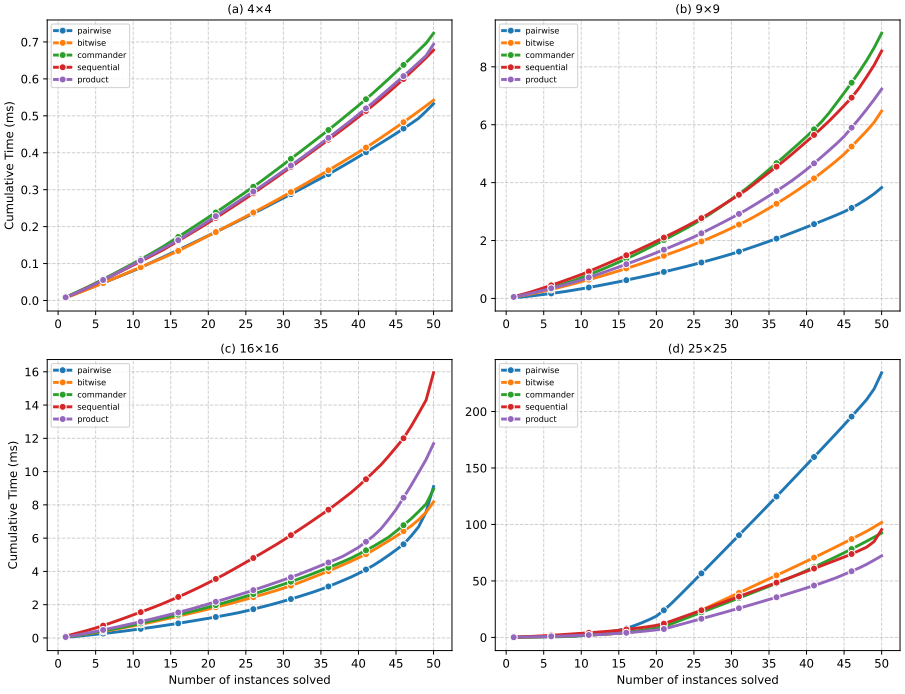
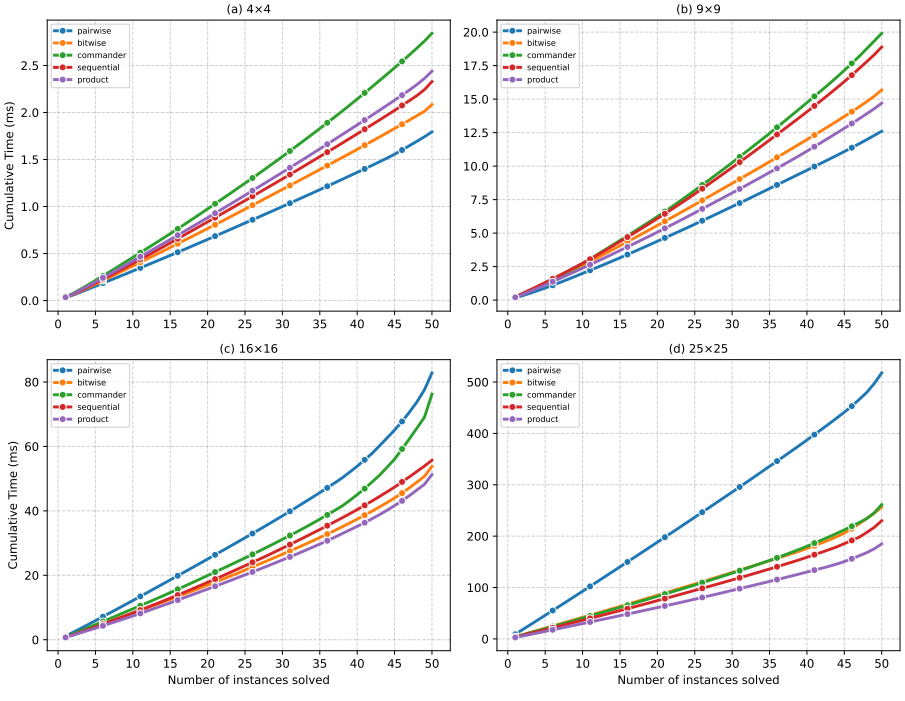


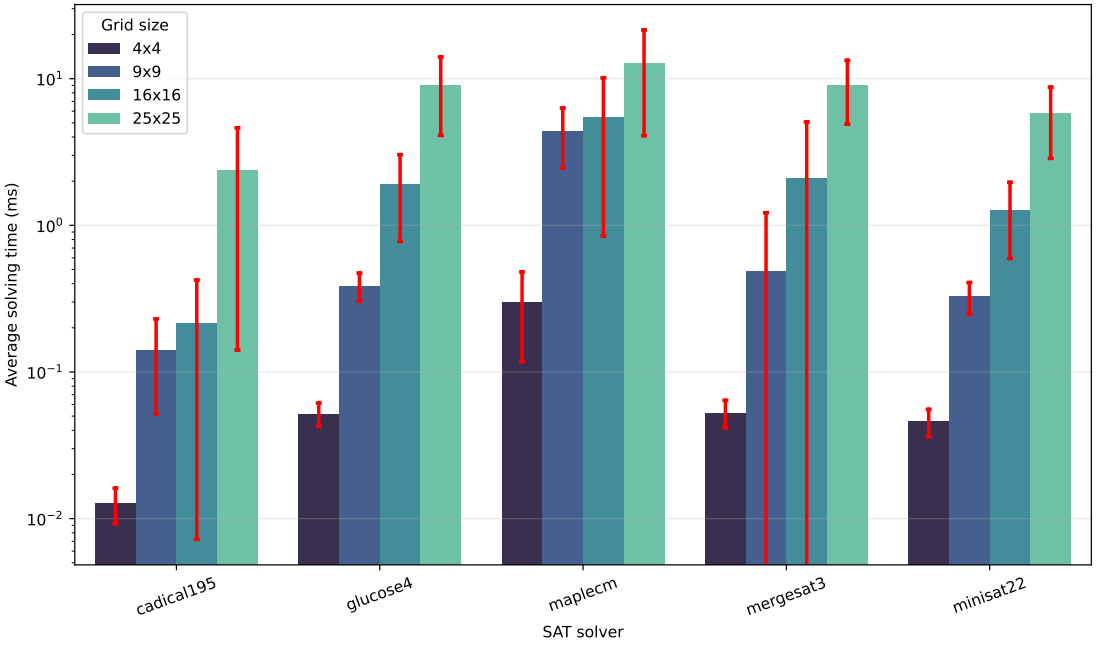
Figure 2. Cumulative solving time by AMO encoding of `Minisat22`, across: (a) 4×4, (b) 9×9, (c) 16×16, (d) 25×25.



I notice a clear crossover as the grid size increases, under both solvers. With `Cadical195`, Pairwise encoding handles small sizes very well—it is the fastest encoding on 4×4 and 9×9 even though it creates the most clauses. However, a crossover happens on 25×25 when it becomes the slowest (4.69,ms vs. 1.44,ms for Product, a 3.3× difference). I see the same thing under `Minisat22` (10.36,ms vs. 3.70,ms, a 2.8× difference). This result on two different solvers tells us that the crossover relates to the encodings' complexity (Section 3.2)—specifically, Pairwise's $\mathcal{O}(n^4)$ clause growth versus the linear or log-linear growth of the other four encodings. Figures 1 and 2 show this clearly: the Pairwise curve (blue) is at the leftmost position in each part (a) – (b) but is the clear rightmost, slowest curve in part (d).

Among the other encodings, Product is the fastest on 25×25 under both solvers. It always beats Commander and Sequential even though they have similar or fewer clauses on this grid size. Based on this, I assume that Product's tighter auxiliary structure—$\mathcal{O}(n)$ auxiliary variables versus $\mathcal{O}(n^2)$ for Sequential and $\mathcal{O}\left(\frac{n^2}{c}\right)$ for Commander—gives a real advantage, not just about the number of clauses. Therefore, I carry forward the *product encoding*, solved with `Cadical195` and `Minisat22`, as the representative SAT-based methods for the comparison with ILP and CP methods on Group M in Section 4.3.

Figure 3. Average solving time per SAT solver, grouped by grid size (averaged over all five AMO encodings; error bars denote standard deviation).



In Figure 3, I compare the average solving time of the five SAT solvers across all five AMO encodings. `cadical195` is always the fastest solver at every grid size, and the gap becomes wider on larger grids. The other four solvers (`glucose4`, `maplecm`, `mergesat3`, `minisat22`) have similar results, though `maplecm` is a bit slower on the two largest grids. This tells us that the choice of solver does not affect the time as much as the choice of AMO encoding, which has a bigger impact when the grid size increases.

Also, I notice that `minisat22` is the most stable solver among the other four. Even though it is the second fastest on average (after `cadical195`), Figure 3 shows that the error bar of this solver is quite short across all four grid sizes, and I can clearly see both its upper and lower bounds on the log scale. This means its solving time is very stable across different puzzles. On the other hand, solvers like `maplecm` can sometimes be as fast as `cadical195`, but they are not stable at all. Based on this, if I want stable performance rather than just the fastest speed, `minisat22` is a much more reliable choice.

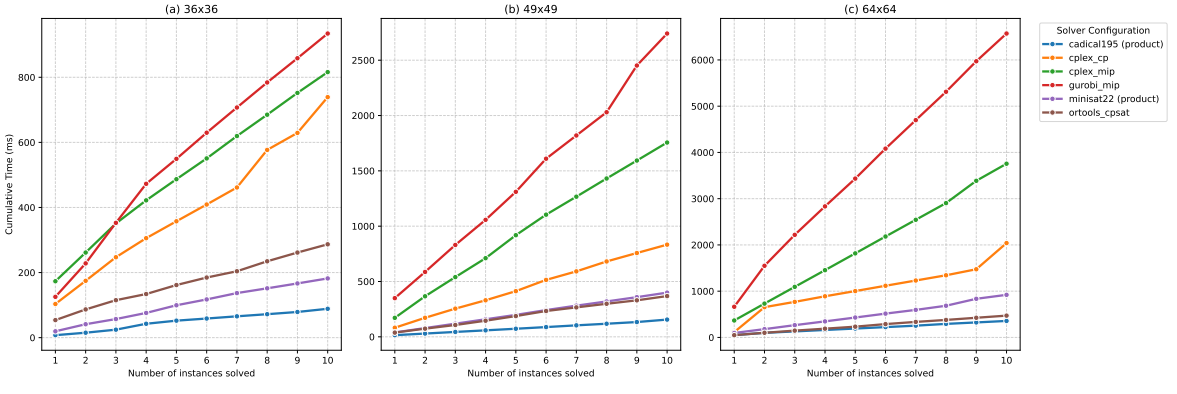
4.3. Comparison with ILP and CP Methods

Based on Section 4.2, I carry forward the product encoding, solved with `Cadical195` and `Minisat22`, as the representative SAT-based methods on Group M. I compare it against two ILP solvers (Gurobi, CPLEX MIP) and two CP solvers (CPLEX CP, OR-Tools CP-SAT) solving the formulations of Section 2.1. Table 6 reports the average and median solving time for each method and grid size.

Table 6. Average and median solving time (ms) on Group M, for each methods.

Grid size	Method	Avg. time (ms)	Median time (ms)
36×36	SAT (Product, Cadical195)	8.87	7.58
	SAT (Product, Minisat22)	18.20	18.38
	OR-Tools CP-SAT	28.68	27.24
	CPLEX CP	73.90	64.87
	CPLEX MIP	81.59	67.83
	Gurobi	93.44	78.87
49×49	SAT (Product, Cadical195)	15.58	14.80
	SAT (Product, Minisat22)	39.94	39.74
	OR-Tools CP-SAT	36.82	36.86
	CPLEX CP	83.31	82.33
	CPLEX MIP	175.60	171.20
	Gurobi	274.11	248.51
64×64	SAT (Product, Cadical195)	35.65	31.48
	SAT (Product, Minisat22)	92.09	85.08
	OR-Tools CP-SAT	47.13	46.77
	CPLEX CP	204.03	114.56
	CPLEX MIP	375.60	363.93
	Gurobi	657.09	632.06

Figure 4. Cumulative solving time by solvers across: (a) 36×36, (b) 49×49, (c) 64×64.



Across all three grid sizes, both SAT-based methods are always among the three fastest methods, no matter which SAT solver is used. Even `Minisat22`, the weaker solver in Section 4.2, has an average time about 2–7× lower than CPLEX CP, CPLEX MIP, and Gurobi at every size (e.g., 18.20 ms vs. 73.90 ms for CPLEX CP at 36×36). This shows that the advantage of the SAT-based approach does not depend on using the fastest available SAT solver, and its performance stays stable. The cumulative curves in Figure 4 show the same trend: the SAT-based lines grow slowly, while the ILP lines rise steeply as the grid gets larger. For example, after 10 instances at 64×64, Gurobi has used about 6,570 ms, whereas `Cadical195` needs only about 360 ms.

The difference is even clearer when we compare the two ILP solvers (Gurobi and CPLEX MIP) with the SAT-based methods. The gap between `Cadical195` and the ILP solvers grows with the grid size—from 9–11× at 36×36

to 11–18× at 64×64—which is consistent with the previous finding that MIP solvers scale poorly on large or highly combinatorial CSPs [17], such as Sudoku. The two CP solvers behave differently. CPLEX CP stays about 5–8× slower than `Cadical195` at all three sizes. Its average time is also less stable: at 64×64 the average (204.03 ms) is much higher than the median (114.56 ms), and its cumulative curve has sudden jumps, which means a few hard instances take much longer than the others. OR-Tools CP-SAT is the opposite: its gap to `Cadical195` shrinks from 3.23× at 36×36 to only 1.32× at 64×64, the smallest gap among all non-SAT methods at any size. At 49×49 and 64×64 it is even faster than `Minisat22` (e.g., 47.13 ms vs. 92.09 ms at 64×64). This suggests that CP-SAT, whose core is built on a SAT solver, benefits from the same techniques as the SAT-based methods, so it scales better than the ILP solvers and CPLEX CP as the grid grows. This raises a question: does the gap between `Cadical195` and OR-Tools CP-SAT keep shrinking, or even reverse, on larger grids? Section 4.4 answers this question in more detail.

4.4. Results on Larger Grids

Based on Section 4.3, I analyze the solvability of SAT-based methods and OR-Tools CP-SAT on big size grids. In this task, `Cadical195` are used to represent SAT-based methods on group L. Two AMO encodings, Commander and Sequential, are also included for comparison to investigate whether the top-2 nearly fastest results in Figures 1 and 2 at (d) 25×25 perform well. Table 7 reports the average and median solving time for each method and grid size.

Table 7. Average and median solving time (ms) on Group M, for each methods.

Grid size	Method	Avg. time (ms)	Median time (ms)
81×81	SAT (Sequential)	78.39	77.30
	SAT (Commander)	84.50	84.56
	SAT (Product)	60.31	58.14
	CP-SAT (OR-Tools)	69.19	65.16
100×100	SAT (Sequential)	240.60	229.56
	SAT (Commander)	184.82	180.73
	SAT (Product)	116.90	113.98
	CP-SAT (OR-Tools)	86.59	82.35

Among the three encoding schemes, Product is still the best, with the smallest average time at both grid sizes. At 81×81, SAT (Product) is the fastest method overall, followed by OR-Tools CP-SAT, and the gap between them is only 8.88 ms. At 100×100, however, we see something interesting: OR-Tools CP-SAT becomes the fastest method, and SAT (Product) moves to second place, 30.31 ms behind. So the order of the two methods is reversed, and the gap between them is about 3.4× larger than at 81×81. The growth of the average time also shows this change. From 81×81 to 100×100, the time of OR-Tools CP-SAT increases by only 1.25× (69.19 ms to 86.59 ms), while SAT (Product) increases by 1.94×, SAT (Commander) by 2.19×, and SAT (Sequential) by 3.07×. The median time of OR-Tools CP-SAT is also very close to its average time (82.35 ms vs. 86.59 ms at 100×100), so this result is not caused by a few unusual instances. Based on this, I think OR-Tools CP-SAT works in a similar way to the SAT-based methods but in a more optimized version, and this is why it scales better on larger grids. Since we only test two sizes here, it is likely, but not yet certain, that OR-Tools CP-SAT stays the fastest method beyond 100×100.

5. Conclusion

For the SAT-based methods, Pairwise encoding worked very well on small grids such as 4×4 and 9×9. This is shown in parts (a) and (b) of Figures 1 and 2 for the two fastest SAT solvers, `Cadical195` and `Minisat22`. However, a crossover appeared as the grid size increased: at 16×16 and 25×25, Product became the fastest encoding scheme, which is probably related to the number of auxiliary variables. The only exception was `Cadical195`, where the error bar in Figure 3 was quite wide, which means its result was not stable in this case. Based on these preliminary tests, I selected Product encoding, evaluated with both `Cadical195` and `Minisat22`, for the comparison with ILP and CP methods.

In this comparison, Table 6 showed that SAT (Product, `Cadical195`) had the fastest average solving time at all three sizes. At 49×49 and 64×64, it was followed by OR-Tools CP-SAT and then SAT (Product, `Minisat22`), while at 36×36 `Minisat22` was the second fastest. The SAT-based methods clearly performed better than the ILP solvers

and CPLEX CP. I also noticed that OR-Tools CP-SAT gradually narrowed the gap with SAT (Product, `Cadical195`), from 19.81 ms at 36×36 to 11.48 ms at 64×64. So I tested them on larger grids, 81×81 and 100×100, and Table 7 showed an interesting result: OR-Tools CP-SAT was faster than SAT (Product) at 100×100, with a gap of about 30.31 ms. Based on this result, I suppose that OR-Tools CP-SAT may become the fastest method on larger grids.

Data availability

The source code, the benchmark puzzles, and the raw experimental results (one CSV row per puzzle, AMO encoding, and solver) are available at `https://github.com/NGKKKKK/N-SQUARE-SUDOKU`.

References

[1] Nysret Musliu and Felix Winter. A hybrid approach for the sudoku problem: Using constraint programming in iterated local search. *IEEE Intelligent Systems*, 32(2):52–62, 2017.

[2] Andrew Bartlett, Timothy P Chartier, Amy N Langville, and Timothy D Rankin. An integer programming model for the sudoku problem. *Journal of Online Mathematics and its Applications*, 8(1):1798, 2008.

[3] Tjark Weber. A sat-based sudoku solver. In *LPAR*, pages 11–15, 2005.

[4] Inês Lynce and Joël Ouaknine. Sudoku as a sat problem. In *AI&M*, 2006.

[5] Carsten Sinz. Towards an optimal cnf encoding of boolean cardinality constraints. *Principles and Practice of Constraint Programming*, pages 827–831, 2005.

[6] Will Klieber and Gihwon Kwon. Efficient cnf encoding for selecting 1 from n objects. In *Proceedings of the Fourth Workshop on Constraints in Formal Verification*, 2007.

[7] Jingchao Chen. A new sat encoding of the at-most-one constraint. In *Proceedings of the Tenth International Workshop on Constraint Modelling and Reformulation*, 2010.

[8] Alan M. Frisch and Paul A. Giannaros. Sat encodings of the at-most-k constraint: Some old, some new, some fast, some slow. In *Proceedings of the Tenth International Workshop on Constraint Modelling and Reformulation*, 2010.

[9] Gurobi Optimization, LLC. Gurobi optimizer reference manual, 2024.

[10] IBM. IBM ILOG CPLEX Optimization Studio, 2023. Version 22.1.

[11] Laurent Perron and Vincent Furnon. Or-tools. *Google*, 2019. CP-SAT solver.

[12] Laurence A. Wolsey. *Integer Programming*. Wiley, Hoboken, NJ, 2nd edition, 2020.

[13] Jean-Charles Régin. A filtering algorithm for constraints of difference in csps. In *Proceedings of the Twelfth National Conference on Artificial Intelligence (AAAI)*, pages 362–367, 1994.

[14] Pham Quang Minh, Dao Xuan Nghia, and To Van Khanh. Exact sat solving for the two-dimensional bandwidth minimization problem. *arXiv preprint arXiv:2608.18514*, 2026.

[15] Alexey Ignatiev, Antonio Morgado, and Joao Marques-Silva. PySAT: A Python toolkit for prototyping with SAT oracles. In *Theory and Applications of Satisfiability Testing – SAT 2018*, volume 10929 of *Lecture Notes in Computer Science*, pages 428–437, 2018.

[16] Mocha2007. Sudoku generator tool. `https://mocha2007.github.io/tools/sudoku.html`.

[17] Neil Newman, Alexandre Fréchette, and Kevin Leyton-Brown. Deep optimization for spectrum repacking. *Communications of the ACM*, 61(1):97–104, 2017.